

MMIS-URA-Revenue Services Workflow & Documentation

UGANDA MARKET MASTER - MMIS Payment Module Holistic Plan & Workflow Focus: Revenue-Service + Payment Package + URA-EFRIS Integration

URA-EFRIS error handling, MTN MoMo webhook, VAT compliance, supplier TIN rules, rent+tax flows, and full monorepo context) consolidate into a **unified production-grade Payment Module**.

Key insights:

- **URA-EFRIS** is mandatory for all taxable supplies (rent, sales, deliveries). MMIS acts as **S2S aggregator** — not every vendor needs individual EFRIS submission if below threshold, but TIN validation + PRN remittance is central.
- **Tax points:** Delivery (supplier → market), Rent (vendor → market authority), POS/Sales (customer → vendor).
- **Delinquency & Lockdown:** 90-day auto-lock on facilities + vendor suspension.
- **Payment Gateways:** MTN MoMo (primary), Flutterwave/Airtel (secondary), bank transfers.
- **Revenue Split:** Market keeps rent + fees; VAT/Tax is collected and remitted to URA via PRN (daily batch).
- **Roles:** Vendor (pays rent+VAT), Market Master (receives rent, configures accounts), Pseudo-Admins (collect at gate/stock counter), Super Admin (national reconciliation).

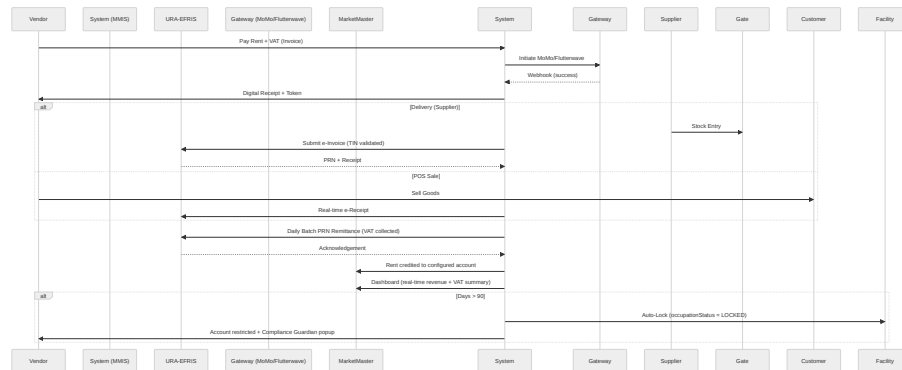
Payment Module Architecture (Conceptual/Ideal)

text

```
1 market-master-mmis/
2 |   apps/
3 |   |   revenue-service/      ← Core Payment + URA-EFRIS logic
4 |   |   |   payment/ (shared package) ← SDK for MoMo, Flutterwave, EFRIS
5 |   |   |   client
6 |   |   |   |   web/ (Next.js)      ← UI + real-time status
7 |   |   |   |   packages/
8 |   |   |   |   |   database/ (Prisma) ← All payment models
9 |   |   |   |   |   infrastructure/ (webhooks)
```

Core Prisma Models (Enhanced from provided schemas)

- Invoice (rent, tax, service)
- Transaction (payment gateway record)
- TaxCollection (EFRIS submission + PRN)
- RentPayment (with delinquency stage)
- Facility (lock status, occupancy)
- Vendor (TIN, delinquencyLevel, daysOverdue)
- MarketToken (payment confirmation token)



Holistic Payment Workflow (End-to-End)

Detailed Workflow Steps (Production Grade)

1. Vendor Pays Rent

- Invoice generated in revenue-service (monthly/quarterly).
- Vendor selects MTN MoMo or Flutterwave.
- Webhook updates RentPayment + Transaction.
- EFRIS e-Receipt issued if VAT applicable.

2. Tax Collection at Multiple Points

- **Delivery:** Gate/Stock counter triggers TaxCollection on entry.
- **Rent:** Bundled in rent invoice.
- **POS:** Real-time during sale.
- TIN validated via URA API (fallback queue on 429/5xx).

3. Market Master Configuration

- Super Admin / National Admin configures Market.revenueAccount (bank or mobile money).
- Rent flows directly to this account after gateway success.
- Market Master sees split: **Rent (kept)** vs **VAT (remitted to URA)**.

4. Government Payment (URA)

- Daily cron in revenue-service aggregates VAT.
- Submits via EFRIS S2S with digital certificate rotation.
- PRN returned and stored for audit.

5. Delinquency & Compliance

- 10/30/60/90-day escalations via DelinquencyNotice.
- 90+ days: Auto-lock facility + vendor suspension.
- Compliance Guardian popup blocks vendor actions until resolved.

Specific Answers to Your Queries

1. How tax is paid to government for a specific user

- Centralised via MMIS as aggregator.
- Vendor pays VAT-inclusive amount → MMIS submits e-Invoice to URA-EFRIS → PRN generated → Daily batch remittance.

- Audit trail in TaxCollection + Transaction.

2. How specific user pays rent and to which account

- Vendor pays via MTN MoMo/Flutterwave to **Market.revenueAccount** (configured by Market Master).
- System splits: Rent → Market account, VAT → URA PRN.

3. If market owner is government → vendor rent + VAT + tax

- Vendor pays combined invoice (rent + VAT).
- MMIS keeps rent portion, remits VAT portion to URA.
- Government (as owner) receives rent directly via configured account.

4. How Market Master gets payment & account configuration

- Market Master (or Super Admin) configures Market.revenueAccount (mobile money or bank).
- All successful rent payments credited in real-time.
- Dashboard shows live revenue + pending VAT remittance.

5. How vendors configure tax & other details

- During KYC/Onboarding: TIN, VAT registration, business license uploaded.
- Vendor dashboard → Profile → Tax Details (editable with admin approval).
- System auto-validates TIN against URA API.

Similar Relative Queries to Consider

- How does the system handle partial payments and instalments?
- What happens on gateway failure (retry + fallback)?
- How are refunds processed with EFRIS reversal?
- Can Market Master configure different commission rates per payment gateway?
- How does offline payment (cash at counter) get reconciled with EFRIS?

☐ INCLUDE ANSWERS:

Full Technical Integration Plan + Code Structure (Revenue-Service Microservice):

```
market-master-mmis/
├── apps/
│   ├── revenue-service/    ← NEW: EFRIS Aggregator Logic
│   │   ├── src/
│   │   │   ├── efris/      ← Core S2S Client
│   │   │   │   ├── efris.service.ts
│   │   │   │   ├── efris.controller.ts
│   │   │   │   ├── dto/    ← EFRIS payloads
│   │   │   │   ├── certificate/ ← Rotation + storage
│   │   │   │   └── error-strategy.ts
│   │   └── payment/       ← Existing MTN MoMo + Flutterwave
│   ├── web/              ← Frontend
│   └── api-gateway/       ← Routes to revenue-service
├── packages/
│   └── shared/            ← EFRIS types + Prisma extensions
├── prisma/
│   └── schema.prisma      ← Updated TaxCollection + PRN
```

Key Prisma Extensions (add to schema.prisma)

```
model TaxCollection {
  id          String @id @default(cuid())
  vendorId    String
```

```

invoiceId      String
prn            String? // Payment Registration Number
status        TaxStatus // PENDING, SUBMITTED, REMITTED, FAILED
efrisResponse  Json?
submittedAt    DateTime?
remittedAt     DateTime?
errorCode      String?
retryCount     Int    @default(0)
certificateUsed String?
// relations...
}

enum TaxStatus {
  PENDING
  SUBMITTED
  REMITTED
  FAILED
  REJECTED
}

```

Error Strategy (Schema – enhanced):

- 429 → Exponential backoff
- Invalid signature → Auto re-certify
- TIN mismatch → Vendor alert + KYC lock

Frontend Payment Flow with Aggregator Branding (Next.js 15 – web/app)

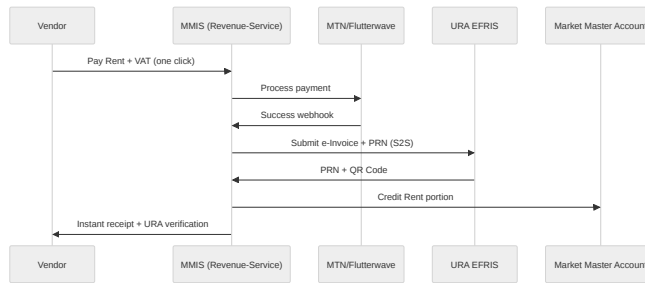
User Flow (Vendor → MMIS → URA)

1. Vendor opens **Payment Dashboard** (branded “MMIS – Official URA EFRIS Aggregator”)
2. Selects Rent + VAT bundle → Auto-generates e-Invoice
3. Pays via **MTN MoMo / Flutterwave / Bank** (one transaction)
4. MMIS splits:
 - Rent → Market Master Account
 - VAT/Tax → Daily batch PRN to URA
5. Instant receipt with **URA QR + Verification Code + Fiscal Number**
6. Dashboard shows live status: “Submitted to URA ✓ Remitted ✓”

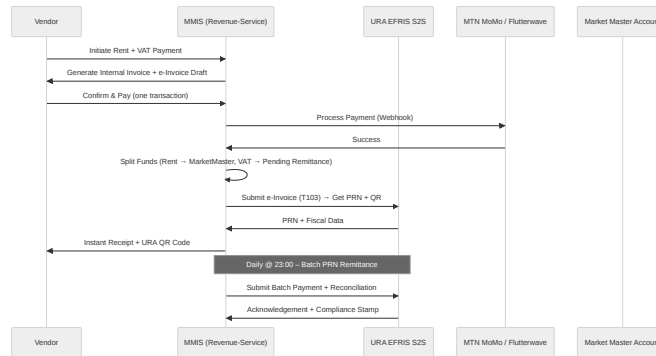
UI Components (shadcn/ui + Tailwind)

- EfrisAggregatorBadge.tsx (green “URA Certified Aggregator”)
- PaymentFlowStepper.tsx with real-time WebSocket updates
- InvoicePreviewModal.tsx showing JSON → rendered e-Invoice
- Branding banner: “Powered by Uganda Market Master – Centralized EFRIS Aggregator”

Mermaid Sequence (End-to-End)



Overall EFRIS S2S Client Workflow (Holistic View)



Tax Collection Points: Rent Payment, Delivery (POS), Manual POS.